



Conception et mise en place d'un système de détection et de réponse aux attaques ransomware dans un environnement contrôlé

Codjo Fréjus Loïc SANGRONIO

Licence Informatique — Option Sécurité Informatique

24 juin 2026

Encadreur :
Ing. Guy KPEDJO

RansomShield

2026-07-05

[OUVERTURE]

Monsieur le Président du jury, Mesdames et Messieurs les membres du jury, bonjour. Je vous remercie pour l'occasion qui m'est donnée de présenter mon travail portant sur la conception et la mise en place d'un système de détection et de réponse aux attaques ransomware dans un environnement contrôlé : RansomShield.



- 1 Introduction Générale
- 2 Revue de littérature
- 3 Analyse et Conception
- 4 Réalisation et Résultats
- 5 Conclusion et Perspectives

Sommaire

- 1 Introduction Générale
- 2 Revue de littérature
- 3 Analyse et Conception
- 4 Réalisation et Résultats
- 5 Conclusion et Perspectives

[SOMMAIRE]

Ma présentation s'articule en cinq parties : le contexte et la problématique, la revue de littérature, la conception, la réalisation et les résultats, puis la conclusion. Je terminerai par une courte démonstration live.

Introduction Générale

2026-07-05

RansomShield
└─ Introduction Générale

Introduction Générale

Contexte

L'évolution rapide du numérique expose les organisations aux ransomwares : des logiciels malveillants qui rendent les données inaccessibles puis exigent une rançon.

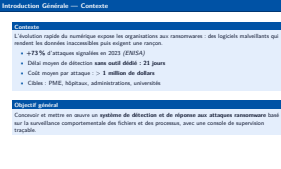
- +73 % d'attaques signalées en 2023 (ENISA)
- Délai moyen de détection sans outil dédié : 21 jours
- Coût moyen par attaque : > 1 million de dollars
- Cibles : PME, hôpitaux, administrations, universités

Objectif général

Concevoir et mettre en œuvre un système de détection et de réponse aux attaques ransomware basé sur la surveillance comportementale des fichiers et des processus, avec une console de supervision traçable.

[CONTEXTE]

Les organisations dépendent aujourd'hui fortement de leurs systèmes informatiques, ce qui les expose de plus en plus aux cyberattaques — en particulier au ransomware. Selon l'ENISA, ces attaques ont augmenté de 73 % en 2023 ; sans outil dédié, il faut en moyenne 21 jours pour les détecter.



La détection comportementale en temps réel des attaques ransomware, combinée à une réponse maîtrisée avec validation humaine systématique, constitue une solution efficace et démontrable pour protéger un parc de machines hétérogène sans déployer de vrai malware.

Défi #1

Variantes inconnues contournent les antivirus par signatures

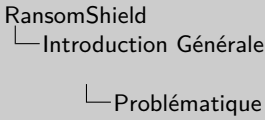
Défi #2

Parc hétérogène (Windows, Linux, macOS) à surveiller

Défi #3

Réponse automatique risquée — contrôle humain indispensable

2026-07-05



[PROBLÉMATIQUE]

La détection comportementale en temps réel des attaques ransomware, combinée à une réponse maîtrisée avec validation humaine systématique, constitue une solution efficace et démontrable pour protéger un parc de machines hétérogène sans déployer de vrai malware. Trois défis structurent ce travail : les variantes inconnues qui contournent les antivirus, la diversité des systèmes à surveiller, et le risque d’une réponse automatique sans contrôle humain.

Problématique

La détection comportementale en temps réel des attaques ransomware, combinée à une réponse maîtrisée avec validation humaine systématique, constitue une solution efficace et démontrable pour protéger un parc de machines hétérogène sans déployer de vrai malware.

Défi #1

Variantes inconnues contournent les antivirus par signatures

Défi #2

Parc hétérogène (Windows, Linux, macOS) à surveiller

Défi #3

Réponse automatique risquée — contrôle humain indispensable

Objectifs spécifiques

- Surveiller les événements fichiers en temps réel sur les terminaux
- Analyser les comportements et calculer un score de risque configurable
- Générer automatiquement des alertes et des incidents de sécurité
- Proposer et exécuter des actions de protection avec **contrôle humain**
- Sécuriser la console par une authentification forte (2FA TOTP RFC 6238)
- Valider le pipeline à l'aide de scénarios d'attaque représentatifs

2026-07-05

RansomShield
└─ Introduction Générale

└─ Introduction Générale — Objectifs spécifiques

Introduction Générale — Objectifs spécifiques
Objectifs spécifiques • Surveiller les événements fichiers en temps réel sur les terminaux • Analyser les comportements et calculer un score de risque configurable • Générer automatiquement des alertes et des incidents de sécurité • Proposer et exécuter des actions de protection avec contrôle humain • Sécuriser la console par une authentification forte (2FA TOTP RFC 6238) • Valider le pipeline à l'aide de scénarios d'attaque représentatifs

[OBJECTIFS]

L'objectif général est de concevoir et mettre en œuvre un système de détection et de réponse aux attaques ransomware, basé sur la surveillance comportementale des fichiers et des processus, avec une console de supervision traçable. Pour y parvenir, six objectifs spécifiques ont structuré ce travail : surveiller les événements fichiers en temps réel, analyser les comportements et calculer un score de risque configurable, générer automatiquement les alertes, soumettre chaque action de protection à une validation humaine, sécuriser la console par une authentification forte à deux facteurs, et valider la solution sur des scénarios d'attaque représentatifs.

Revue de littérature

2026-07-05

RansomShield
— Revue de littérature

Revue de littérature

Définition d'un ransomware

Un ransomware est un logiciel malveillant qui vise à bloquer l'accès à un système ou à des données, souvent par chiffrement, puis à réclamer une rançon à la victime. (NIST SP 1800-25, CISA)

Comportements caractéristiques — détectables avant les dommages irréversibles

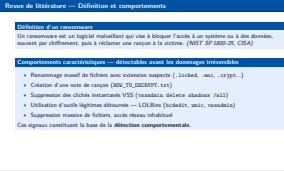
- Renommage massif de fichiers avec extension suspecte (.locked, .enc, .crypt...)
- Création d'une note de rançon (HOW_TO_DECRYPT.txt)
- Suppression des clichés instantanés VSS (vssadmin delete shadows /all)
- Utilisation d'outils légitimes détournés — LOLBins (bcdedit, wmic, vssadmin)
- Suppression massive de fichiers, accès réseau inhabituel

Ces signaux constituent la base de la **détection comportementale**.

2026-07-05

[DÉFINITION]

Un ransomware laisse des traces comportementales avant que les dégâts soient irréversibles : extensions suspectes, note de rançon, suppression VSS, LOLBins. Ce sont ces signaux qui forment la base de notre détection.



Solutions existantes

- **Antivirus** — détection par signatures statiques (ClamAV, Windows Defender)
- **SIEM génériques** — collecte de journaux sans corrélation comportementale
- **EDR commerciaux** — CrowdStrike Falcon, SentinelOne, Microsoft Defender XDR

Limites des solutions existantes

- ❌ Contournables par variantes nouvelles, obfuscation ou LOLBins
- ❌ Coût élevé des EDR commerciaux (hors portée académique)
- ❌ Absence d’outil open-source académique démontrable en conditions réelles

Notre réponse : RansomShield

Approche comportementale open-source, multi-plateforme, démontrable sur machines virtuelles réelles — **contrôle humain à chaque étape.**

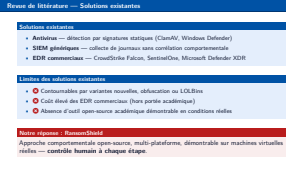
RansomShield
└─ Revue de littérature

└─ Revue de littérature — Solutions existantes

2026-07-05

[SOLUTIONS]

Les antivirus ne couvrent que les variantes connues par leur signature. Les SIEM collectent des journaux, mais sans corrélation comportementale. Les EDR commerciaux sont efficaces, mais leur coût les rend inaccessibles dans un cadre académique. RansomShield occupe cet espace : une solution open-source, comportementale, multi-plateforme, démontrable sur machines réelles, avec contrôle humain intégré.

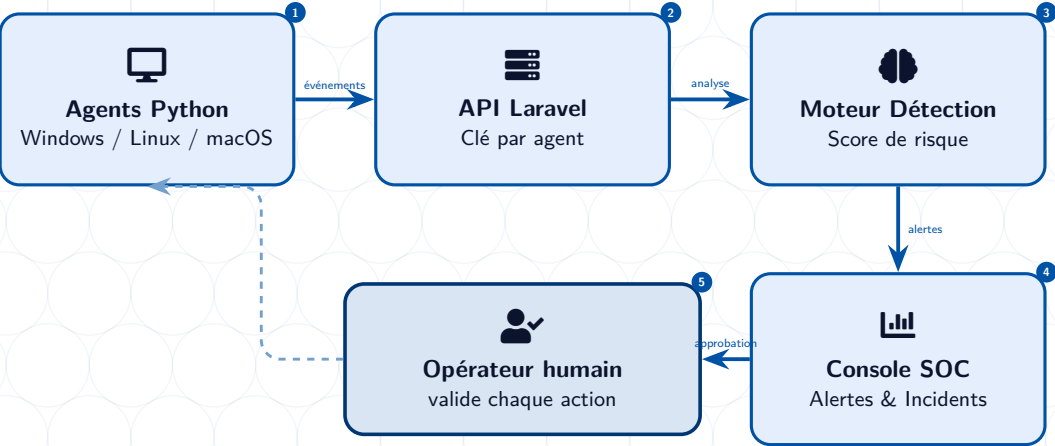


2026-07-05

RansomShield
└ Analyse et Conception

Analyse et Conception

Analyse et Conception

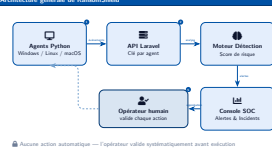


Aucune action automatique — l'opérateur valide systématiquement avant exécution

2026-07-05

RansomShield
Analyse et Conception

Architecture générale de RansomShield



[ARCHITECTURE]

L'architecture suit un pipeline en cinq maillons. L'agent collecte les événements sur la machine et les envoie à l'API Laravel. Le moteur de détection analyse les signaux et calcule un score de risque. Dès que ce score franchit un seuil, la console SOC génère une alerte et soumet une action à l'opérateur. C'est lui qui décide — le système n'agit jamais de manière autonome.

 Agent Python

Surveille les dossiers via **Watchdog** / **inotify**. File locale SQLite (résilience réseau). Enrôlement par token à usage unique. Service `systemd` — démarrage automatique.

 Découverte réseau

fping (ICMP) + ARP sweep sur les /24. Scan automatique toutes les 5 minutes. Aucun enrôlement sans **validation manuelle admin**.

 Console SOC Laravel

Dashboard temps réel (Chart.js, AJAX 30s). Alertes, incidents, timeline, file d'approbation. Notifications : web, e-mail, son, webhook.

 Module de simulation

5 scénarios d'attaque, jusqu'à **22 événements**. Tous marqués `is_simulation=true` — nettoyage facile après chaque démo.

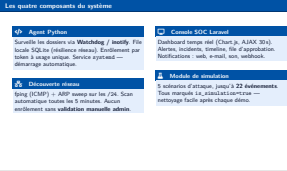
2026-07-05

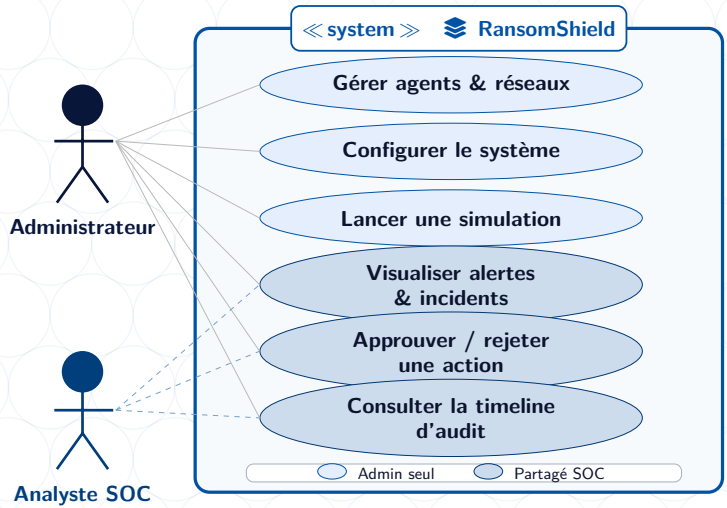
RansomShield
└─ Analyse et Conception

└─ Les quatre composants du système

[4 COMPOSANTS]

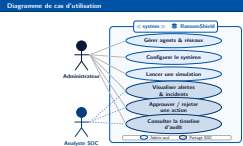
RansomShield repose sur quatre composants distincts. L'agent Python surveille les fichiers en temps réel et maintient une file locale SQLite pour ne perdre aucun événement en cas de coupure réseau. La console SOC centralise la supervision, les alertes et la file d'approbation. Le module de découverte réseau détecte les nouvelles machines, mais aucune n'est enrôlée sans validation manuelle de l'administrateur. Le module de simulation rejoue des scénarios d'attaque réalistes, sans déployer de vrai malware.





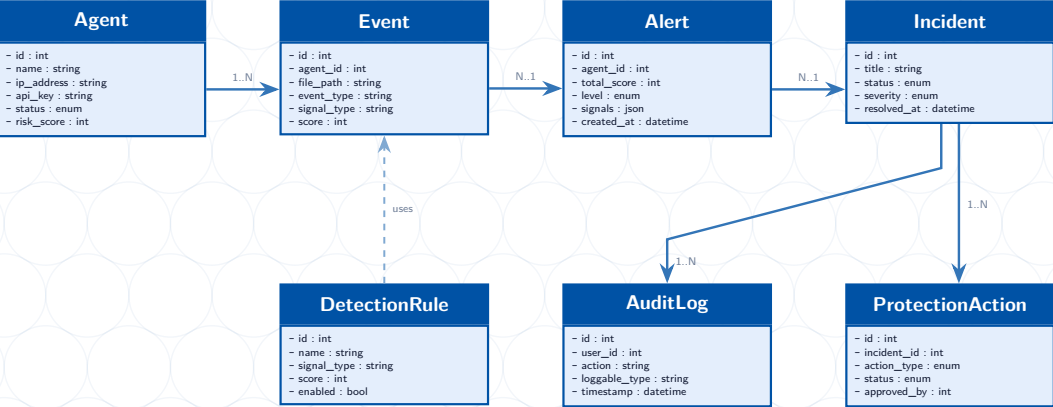
2026-07-05

RansomShield
└ Analyse et Conception
└ Diagramme de cas d'utilisation



[USE CASE]

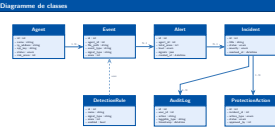
Deux acteurs interagissent avec le système. L'administrateur gère le parc : il enrôle les agents, configure les règles de détection et lance les simulations. L'analyste SOC traite les incidents au quotidien : il consulte les alertes, approuve ou rejette les actions de protection, et suit la timeline d'audit. Il n'a pas accès à la configuration — cette séparation est intentionnelle.



2026-07-05

RansomShield
└ Analyse et Conception

└ Diagramme de classes



[DIAGRAMME DE CLASSES]

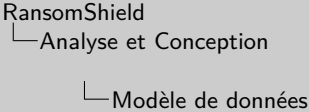
Sept entités structurent le modèle objet. Le pipeline se lit de gauche à droite : un Agent génère des Events que les DetectionRules évaluent pour produire des Alerts. Ces alertes s'escaladent en Incidents, qui génèrent des ProtectionActions soumises à approbation, et des entrées dans l'AuditLog pour la traçabilité complète.

Tables principales	
agents	enrôlement, état, risque
events	événements fichiers
alerts	score, niveau, signaux
incidents	fiche, statut, timeline
detection_rules	règles configurables
protection_actions	décisions SOC
audit_logs	traçabilité horodatée
simulation_runs	profils d'attaque

Relations clés	
agents	→ events (1-N)
events	→ alerts (N-1)
alerts	→ incidents (N-1)
incidents	→ protection_actions (1-N)
incidents	→ audit_logs (1-N)
simulation_runs	→ events (1-N)

8 tables MySQL — pipeline complet :
événement → alerte → incident → action → audit

2026-07-05



[MODÈLE DE DONNÉES]

Le modèle de données traduit directement le pipeline de traitement. Huit tables MySQL couvrent l'ensemble du cycle, de la collecte des événements jusqu'à l'archivage des décisions. La traçabilité est intégrée au schéma : chaque action de protection est liée à l'identité de l'opérateur qui l'a validée et conservée dans l'audit log.

2026-07-05

RansomShield
— Réalisation et Résultats

Réalisation et Résultats

Réalisation et Résultats

Backend & Base de données

- Laravel 11 / PHP 8.3
- MySQL 8.0 (Eloquent ORM)
- API REST sécurisée (clé per-agent)

Frontend

- Blade (templates Laravel)
- Chart.js (intégré local)
- AJAX toutes les 30 s

Agent de surveillance

- Python 3.10
- Watchdog (inotify Linux)
- SQLite (file locale offline)
- psutil (processus)

Infrastructure de test

- 3 VMs KVM/libvirt
- Réseau 10.20.0.0/24
- 2FA TOTP RFC 6238
- Token usage unique (enrôlement)

[CHOIX TECHNIQUES]

Les choix techniques ont été guidés par trois critères : la portabilité, la légèreté d'installation, et la capacité à démontrer le système en conditions réelles. Laravel 11 pour l'API REST et la console, Python avec Watchdog pour l'agent multi-plateforme, Chart.js intégré en local pour le frontend, et un environnement de test sur trois machines virtuelles KVM avec authentification 2FA TOTP.



Cycle de vie de l'agent

- 1 Démarrage : lecture .env
- 2 Enrôlement : token → clé API
- 3 Watchdog observe les dossiers
- 4 Événement → file SQLite locale
- 5 POST /api/agent/events
- 6 Heartbeat toutes les 30 s

Installation en une commande :

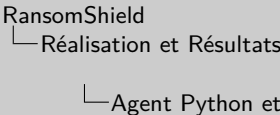
```
curl -sSL http://<soc>/install.sh | bash
```

Scoring comportemental

Signal détecté	Score
Extension suspecte (.locked)	+80
Note de rançon créée	+55
Suppression clichés VSS	+60
LOLBin suspect	+45
Suppression massive fichiers	+50

Score < 30	Normal	
30 à 59	Suspect	
60 à 99	Élevé	Alerte + incident créés automatiquement en < 5 s
≥ 100	Critique	

2026-07-05



Agent Python et moteur de détection

Cycle de vie de l'agent

- 1 Démarrage : lecture .env
- 2 Enrôlement : token → clé API
- 3 Watchdog observe les dossiers
- 4 Événement → file SQLite locale
- 5 POST /api/agent/events
- 6 Heartbeat toutes les 30 s

Installation en une commande :

```
curl -sSL http://<soc>/install.sh | bash
```

Scoring comportemental

Signal détecté	Score
Extension suspecte (.locked)	+80
Note de rançon créée	+55
Suppression clichés VSS	+60
LOLBin suspect	+45
Suppression massive fichiers	+50

Score < 30 Normal

30 à 59 Suspect

60 à 99 Élevé

≥ 100 Critique

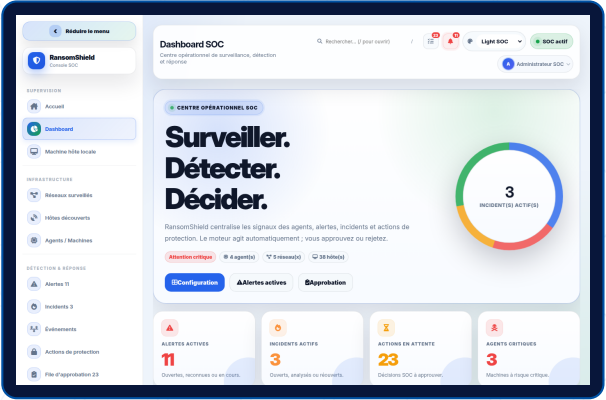
Alerte + incident créés automatiquement en < 5 s

[AGENT]

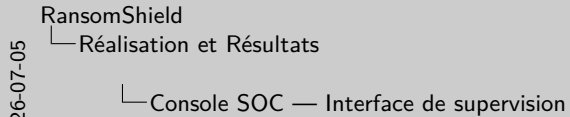
Pour illustrer le scoring : un fichier renommé avec l'extension .locked déclenche un signal à +80, la création d'une note de rançon ajoute +55. Le score cumulé de 135 dépasse le seuil critique fixé à 100 : une alerte est générée et un incident est ouvert automatiquement en moins de cinq secondes.

Pages de la console

- Tableau de bord
- Agents (état, heartbeat)
- Alertes (score, signaux)
- Incidents
- Timeline d’audit
- File d’approbation
- Règles (CRUD live)
- Recherche globale
- Paramètres système

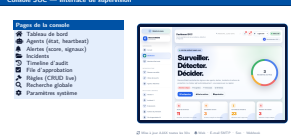


Mise à jour AJAX toutes les 30 s Web · E-mail SMTP · Son · Webhook



[CONSOLE SOC]

La console couvre l’ensemble du cycle opérationnel en neuf pages. Chaque action de protection passe obligatoirement par la file d’approbation : l’analyste valide ou rejette explicitement — aucune exécution silencieuse n’est possible. Les notifications sont envoyées simultanément sur quatre canaux : web, e-mail SMTP, son et webhook.



5 scénarios d'attaque testés

Scénario	Évts	Résultat
Chiffrement basique	7	✓
Kill chain complète	22	✓
Chiffrement massif	18	✓
Exfiltration données	8	✓
LOLBins suspects	5	✓

réseau 10.20.0.0/24

3 VMs KVM —

Résultats obtenus

- ✓ Détection en < 5 secondes (tous scénarios)
- ✓ Aucun événement perdu (file SQLite)
- ✓ Pipeline complet validé bout en bout
- ✓ Rollback isolation réseau fonctionnel
- ✓ 2FA TOTP et API sécurisée (per-agent)

Limite identifiée

Tests conduits sur Linux uniquement. Un attaquant ralentissant son rythme réduit la visibilité.

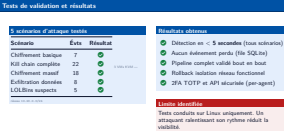
2026-07-05

RansomShield
Réalisation et Résultats

Tests de validation et résultats

[TESTS]

Cinq scénarios d'attaque ont été exécutés, de 5 à 22 événements. Tous ont été détectés en moins de cinq secondes, sans perte d'événement. Le pipeline complet — de la collecte à l'incident — a été validé bout en bout. Une limite est à noter : les tests ont été conduits sur Linux uniquement, ce qui constitue la première perspective d'évolution du projet.





Démonstration Live

2026-07-05

RansomShield
— Réalisation et Résultats

▶ Démonstration Live

[DÉMONSTRATION LIVE]

Suivre le guide de démonstration.

Checklist rapide :

1. Console SOC ouverte — 3 agents en ligne
2. SSH sur `rs-client-1` — renommage `.locked` + note de rançon
3. Revenir sur la console — alerte critique apparue en moins de 5 secondes
4. File d'approbation — approuver l'action, montrer le rollback
5. Timeline — tout est horodaté
6. Simulation kill chain complète — 22 événements

Conclusion et Perspectives

Conclusion

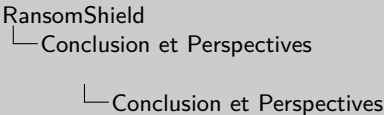
RansomShield est une solution opérationnelle couvrant l'ensemble du cycle de traitement d'un incident : **collecte** → **analyse** → **alerte** → **réponse** → **audit**.

- ✓ Agent Python multi-plateforme — installation en une commande
- ✓ Console SOC : simulation, 4 canaux de notification, rapports PDF, 2FA
- ✓ Moteur de détection : scoring configurable, règles CRUD, < 5 s
- ✓ Environnement contrôlé : aucun vrai malware déployé

Perspectives d'évolution

- **HTTPS** — chiffrement TLS en production
- **RBAC** — rôles distincts (administrateur, analyste SOC)
- **Machine Learning** — Isolation Forest (baseline comportementale)
- **MITRE ATT&CK** — alignement et couverture du référentiel
- **Intégration SIEM** — export Syslog vers Splunk / ELK Stack

2026-07-05



[CONCLUSION]

RansomShield couvre l'intégralité du cycle de traitement d'un incident : collecte, analyse, alerte, réponse et audit. Ses limites sont assumées : les tests ont été conduits sur Linux, et la communication reste en HTTP sur un réseau local contrôlé. Les perspectives visent le chiffrement TLS en production, l'introduction de rôles distincts via RBAC, un moteur de détection par Machine Learning, et un alignement sur le référentiel MITRE ATT&CK.



Merci pour votre attention

2026-07-05

RansomShield
└─ Conclusion et Perspectives

Merci pour votre attention

[QUESTIONS DU JURY]

- **Pourquoi Python et pas C/Go ?** Watchdog exploite inotify nativement, CPU minimal, portable
- **Faux positifs ?** Seuils configurables, opérateur humain valide ou rejette
- **Pourquoi pas HTTPS ?** LAN contrôlé ; 1^{re} perspective pour la prod
- **Différence avec un antivirus ?** Comportement, pas signature ; variantes inconnues couvertes
- **Testé sur Windows ?** Non, limite assumée : Linux uniquement

*Garder les **annexes** prêtes.*